THE EDITOR OF THE SUITE

bm Pixel

Sprites, tilesets and hitboxes: draw the graphics of a bm game on the console.

Contents

What bm Pixel is 4

What it does 4

Why it matters 4

Good to know 4

Caveats 4

Opening a file 5

Where to start 5

Projects and games 5

The menu (Esc) 5

The pages 6

F1: draw 6

F2: the sheet 6

F3: the palette 6

F4: the stack 6

The hitbox preview (j) 7

Gamepad 7

Drawing tools and keys 8

What it does 8

The keys of the drawing page 8

Tile view (q) 9

Shapes and selections 9

Copy, move and transform 9

Export a PNG 9

The assistant (F6) 10

Sprites, zones, frames and fps 11

What it does 11

How bm Pixel shows frames 11

Good to know 11

Caveats 11

Tile flags **12**
What it does 12
The convention 12
Good to know 12
Caveats 12

Hitboxes and hurtboxes **13**
What it does 13
How bm Pixel shows them 13
Caveats 13

Palettes **14**
What it does 14
The keys of F3 14
Good to know 14
Caveats 14

Importing and exporting resources **15**
What it does 15
What you can do today 15
Caveats 15

Guided tutorials **16**
Draw your first sprite 16
Make an animated sprite 16
Mark the tiles that are solid 16
Add a hitbox 17

Tips and limits **18**
Limits 18
Tips 18
When something goes wrong 18

Open points **19**

README

What this is. The manual of **bm Pixel**, the pixel art editor built into the console. It works on the sprite sheet of a game or of a project.

How to use it. Read the first three chapters once, then follow the numbered steps of the tutorials. Anything in a bordered box with a shadow is meant to be copied as it is.

What is missing. Where bm Pixel does not do something yet, or the repository does not say how, the manual shows a highlighted TO FILL tag. Nothing is invented.

What bm Pixel is

bm Pixel is the drawing tool of the bm suite. You draw sprites and tiles on the **sprite sheet**: the one picture that a game reads with `spr`, `sspr` and `map`, and that bm Studio uses as the textures of its models.

What it does

- Draws a sprite enlarged, pixel by pixel, with pencil, eraser, fill, lines, rectangles, ovals, selections and a mirror.
- Plays the sprite and the ones next to it as an animation, with onion skin.
- Shows the whole sheet, lets you pick a sprite, copy it and resize the sheet.
- Edits the palette of up to 256 colours.
- Shows the hitboxes and hurtboxes of the sheet's zones on top of your drawing.

Why it matters

A game looks like what is on its sheet. With bm Pixel you draw on the console itself, try the game with one key (F5) and come back to the same place.

Good to know

- There are four pages: **F1** draw, **F2** sheet, **F3** palette, **F4** stack. **Esc** opens the menu.
- Hold **F12**, or press **?**, to see the keys of the page. **Tab** (on the pad **X**) opens the list of the page's commands.
- With a mouse, the mouse does what the keys do.
- In a dialog line, a click puts the caret between the letters. On the palette page the wheel moves the choice one row; over R, G or B it changes that value.

Caveats

- bm Pixel edits the sheet and the palette. It makes only stack zones (F4). It does not name other zones, draw hitboxes or set tile flags (see the chapters on them).

Opening a file

Where to start

Start from the **Dev** tab of the menu, or press **X** on the cover of a game and choose **Open in bm Pixel**. If you start bm Pixel with no file, it shows the list of the `.bme` projects of `/bme` and the `.bm` and `.bme` files it finds in `/carts` and in the root of the card, and a new empty sheet is ready behind it.

Projects and games

- A **project** is a `.bme` file. The editors change projects and save them where they are.
- A **game** is a `.bm` (or `.b16`) file. It is read only. When you open one, the screen says `a game: saving makes its copy`, and saving writes an editable `.BME` copy, never the game itself.
- Only the sheet and the palette change in the saved file: the code, the map, the cover, the sounds and the models stay as they were.

The menu (Esc)

Entry	Key	What it does
Continue		back to the page
Open...	Ctrl+O	opens another `.bm` or `.bme`
New sheet	Ctrl+N	a new empty sheet of 256x256
Save	Ctrl+S	saves the file
Save as...	Ctrl+Shift+S	asks a file name (8.3, in `/carts`) and saves there
Try the game	F5	saves if needed, runs the game, and brings you back to the same page
Sheet size...		the same as **Shift+R** on the sheet page
Back to bm SDK		returns to bm SDK; it appears only when bm Pixel was opened from the SDK
Exit bm Pixel		leaves; if there are unsaved changes, choose it again to confirm

Ctrl+Z and Ctrl+Y undo and redo. For each file bm Pixel remembers the sprite, its size, the animation, the colour and the page.

> **NOTE**
> A new sheet saved with *Save as* becomes a cartridge whose code just shows the sheet. Replace that code with your game's.

The pages

F1: draw

The chosen sprite, enlarged on the left. On the right: the colours, the tool, the sprite number with its place on the sheet, the sprite as the game draws it, and the animation next to it. A line under them says the speed and whether pause, onion skin, boxes and mirror are on.

[TO FILL: shots/editor-pixel-draw.png]

The sprite size is 8, 16, 32, 64 or 128 pixels: **z** goes to the next size. **PgUp** and **PgDn** go to the sprite before and after. **t** shows or hides the grid.

F2: the sheet

The whole sheet. The arrows choose a sprite (in the chosen size), **Enter** draws it. **+** and **-** zoom. The frames of the animation are outlined after the sprite.

[TO FILL: shots/editor-pixel-sheet.png]

F3: the palette

All the colours, up to 256, with the red, green and blue value of the one chosen. See the chapter on palettes.

F4: the stack

A *stack zone* is a zone with *n* slices of the same size that lie side by side on the sheet, from left to right. Slice 1 is the bottom. In the cartridge it is the line `stack name x y w h n` ; a game draws it with `zstack` . The Stack page shows one slice, zoomed, on the left. On the right it shows the stack zones, a row of small pictures of the slices, and the whole stack as it turns (`stack` with a rotation angle). The stack reads the sheet each frame, so a change to a slice shows at once. A `stackv` zone has the slices one under the other on the sheet, from the top down; the page reads and writes both layouts. The line ends with `h` (side by side, `stack`) or `v` (one under the other, `stackv`).

Key	Action
left / right	the slice before / after
up / down	the stack zone before / after
PgUp / PgDn	8 slices
Enter	draw the slice on F1 (square slices only; **F4** comes back)
k	onion skin: the slice before, dimmed, in the empty pixels of this slice
c	copy this slice to the next one and go to it (extrusion; **Ctrl+Z** undoes)
Del or x	delete the zone (press twice; the pixels stay on the sheet)
n	new zone: name, x, y, w, h, n, `h` or `v` (for example `tree 0 0 16 16 8 h`)

Key	Action
e	change the zone (the same line; a new name makes a new zone)
p	the stack turns or stops
, and .	turn the stack by hand

When no stack zone exists, **n** makes the first one. The line starts with the sprite chosen on F1. **Ctrl+S** writes the zones and the slice count (byte 26 of the zone) with the sheet. There is no undo for the zone line; the copy of a slice has one. The onion skin draws at most 600 small squares, only when it is on. With a mouse: click a zone or a small picture to choose it, double click a small picture to draw it, the wheel changes the slice, a drag on the turning stack turns it by hand, a click on it stops or starts the turn, and the right button opens the list of commands. On the pad: left / right the slice, up / down the zone, **A** draw the slice, **B** turn or stop, **X** + left / right turn by hand, **X** (tap) the list of commands.

The hitbox preview (j)

Press **j** on F1 or F2 to show or hide the boxes of the `BOXES` section of the file. The message says `hitboxes on (BOXES)` or `hitboxes off`. See the chapter on hitboxes.

Gamepad

Keys	Action
Y + left / right	change page (F4 is a page)
Y + B	menu
Y + A	undo
Y + up / down	the sprite before / after
D-pad	the pointer
A	draw (held: a stroke)
B	colour from the sprite, or cancel
X + left / right	the colour
X + up / down	the tool
X (tap)	the list of commands

Drawing tools and keys

What it does

On F1 a pointer moves with the arrows. **Space** uses the tool. With the pencil or the eraser, holding space draws a stroke.

The keys of the drawing page

Key	Action
arrows	move the pointer
space	use the tool
b	pencil
e	eraser
g	fill
i	pick the colour of a pixel (a new colour goes into the palette)
l	line
u / U	rectangle / filled rectangle
o / O	oval / filled oval
m	select a rectangle (it also clears a wand selection)
w	wand select: the area of one colour
q	tile view on / off
, and .	the colour before / after
x	the colour used before
1 - 9, 0	the first ten colours
y	mirror drawing, left-right
t	grid
k	onion skin
p	play or pause the animation
+ / -	more / fewer frames
> / <	faster / slower
z	sprite size
PgUp / PgDn	sprite before / after
j	show or hide the boxes
Ctrl+E	export a PNG or a GIF to `/docs`

Key	Action
F6	the assistant: a base for a sprite
Tab	the list of commands

Colour 0 is transparent.

The hint bar at the bottom of the drawing page shows only the main keys: the tools `b e g m w`, the shapes `l u o`, `Ctrl+E PNG`, the colour keys, **z** and **Tab**. The other keys (`j q` and the rest) are in the **F12** help. The bar ends with the chip F12 keys. Click it, or hold F12, to see every key of the page.

Tile view (q)

Press **q** to see the sprite as a tile. The preview on the right shows the sprite 3 times 3, side by side. You see the seams between the copies. The text under the preview says `tile view 3x3 (q)`. Sprites up to 32x32 fit in the preview. Press **q** again to go back. The menu has the entry *Tile view (q)*.

Shapes and selections

For lines, rectangles, ovals and selections: press **space** to fix one corner, move the pointer to the other corner, press **space** again. **Esc** cancels.

Copy, move and transform

Key	Action
Ctrl+C / Ctrl+X / Ctrl+V	copy, cut, paste (the block floats: move it with the arrows, put it down with space or Enter)
Enter	lift the selection to move it
h / v	flip left-right / up-down
r	turn by a quarter
Del	clear the selection, or the whole sprite
Ctrl+E	export the sprite, the animation or the sheet as a PNG, or the animation as a GIF
Shift+w a s d	scroll the sprite by one pixel (what goes out comes back on the other side)

The transparent pixels of a pasted block leave what is underneath.

Wand select (w)

Press **w** and then **space** on a pixel. bm Pixel selects the connected area of the same colour: the pixels that touch the one you chose, up and down and left and right, and have its colour. A dotted line goes round the area. The status line says how many pixels you chose. Now **Ctrl+C**, **Ctrl+X**, **Del**, **Enter** (lift), **h**, **v** and **r** act only on these pixels. The rest of the sprite stays. Press **m** and select a rectangle to clear the wand selection.

Export a PNG

Press **Ctrl+E** (menu: *Export a PNG or GIF to /docs...*) to write a PNG file or a GIF file. A list asks what to export:

- *this sprite*: the sprite you draw;
- *the animation as a PNG*: its frames side by side in a strip;
- *the animation as a GIF: N frames, F a second*: an animated image, with one image for each frame, in the order of the preview;
- *the whole sheet*.

Then a line asks the file name and suggests one made from the file name. The PNG goes to `/docs` on the card. The pixels with colour 0 are transparent in the PNG. The name is 8.3 in lower case: letters, digits and `_`, and the extension `.png`. If the name has more than 8 letters, bm Pixel cuts it to 8 and the status line says `name cut to 8 letters`. The status line shows `saved /docs/NAME.png` when the file is written. If the export fails, the status line says why.

The GIF has these rules:

- It loops for ever.
- The transparent pixels stay transparent.
- The speed comes from the fps of the sprite, so it is the speed of the preview.
- The file name has up to 8 letters and the extension `.gif`. The file goes to `/docs`.
- A GIF holds 255 colours at most. If the frames have more, bm Pixel reduces them to 255.

The assistant (F6)

F6 asks for a word ("slime", "coin", "ship") and draws a base with the colours of your palette. It floats like a paste: move it, then press Enter. If it is not available, the screen says `the assistant is not here`.

Sprites, zones, frames and fps

What it does

A **zone** is a named part of the sheet: a position, a size, a number of **frames** (1 to 16) and a speed in **fps** (0 is still). A game draws a zone with `zspr("hero_run", x, y)`. The frames are boxes of the same size, side by side on the same row, starting from the first.

How bm Pixel shows frames

- The chosen sprite and the ones that follow it in the sheet are the frames of the animation. **+** and **-** set how many (1 to 16), **>** and **<** set the speed (1 to 30 fps), **p** pauses.
- The box on the right plays them, next to the sprite at actual size.
- When the sprite you are on matches a zone of the file (first pass: the same row, the sprite size equal to the frame size of the zone, and the column anywhere inside the span of the zone's frames, so any of its frames matches; failing that, second pass: the same corner at any size, which shows frame 1), the panel shows `zone "name" frame n/m` instead of the `spr(...)` call.

Good to know

- The frames and the speed you set are remembered for each file by bm Pixel, but they are not written into the cartridge.
- For frames to match a zone, draw them one after the other from left to right.

Caveats

- [TO FILL: how to name a zone and give it frames and fps. The old notes say bm Pixel makes zones, but the code of F1, F2 and F3 has no key for it; today zones are written in a text file (`name x y width height [frames [fps]]`) and passed to `mkbm.py --sprites`]

COPY AND PASTE

```
# zones.txt: name x y width height [frames [fps]]
hero_still   0  32 16 16
hero_run     16 32 16 16 4 10
coin         0  48 8  8  6 12
```

Tile flags

What it does

Every tile of the sheet (every 8x8 cell) has 8 flags, numbered 0 to 7. A game looks at the flags instead of the tile numbers to know what a tile is.

The convention

Flag	Value	Meaning
0	1	solid
1	2	platform: you can pass through from below
2	4	ladder
3	8	water
4	16	hurts
5 - 7		free

This is the convention of bmlib; every game can choose another. The game reads a flag with `fget(n, f)` and the flags under a rectangle with `mflags`.

Good to know

- The flags are saved in the `FLAGS` section of the file, by the place of the cell: a sheet made bigger later keeps them.
- Tiles in an 8x8 grid: draw the tiles at size 8 (**z**) so that one sprite is one tile.

Caveats

- bm Pixel does not show or set flags. **[TO FILL: the exact keys to set a tile's flags on the bm SDK sprite page (the notes say keys 0 to 7 turn the cell's flags on) and how the screen shows them]**

Hitboxes and hurtboxes

What it does

The `BOXES` section holds, for each frame of a zone, the rectangles a game uses in a fight: where the character can be hit, where its blow hurts, the room it takes.

Type	Number	Colour on the screen
hurt: where it can be hit	0	blue
hit: where the blow hurts	1	red
body: the room it takes	2	green
other: the game's own	3 to 255	yellow

A box is given from the top-left corner of the frame, in pixels, and it can stick out of it. A box with frame 0 is valid for every frame of the zone.

How bm Pixel shows them

- **j** turns the boxes on and off (they start on).
- On F1 they are drawn over the enlarged sprite and over the two small previews (the sprite, and the frame the animation is on).
- On F2 they are drawn over every frame of every zone you can see.
- A box shows only when the sprite you are on matches a zone of the file.

[TO FILL: shots/editor-pixel-boxes.png]

Caveats

- You cannot draw, move or resize a box in bm Pixel. [TO FILL: how to create boxes on the console. Today they are written under the zone in the zones text file, with lines `hurt`, `hit` or `body`, the frame (`*` for every frame) and the rectangle x, y, width, height]

COPY AND PASTE

```
punch        0  64 32 32 4 12
  hurt  *    8  2 16 30         # the body, in every frame
  hit   3    24 10 10  6        # the fist comes out in frame 3
```

Palettes

What it does

The palette is the list of the colours you draw with, up to 256. It is saved with the sheet: when the sheet has at most 256 colours it becomes a `SHEET8` section with your palette first, and reopening the file brings the same palette back. Games, the SDK, bm Studio and bm Animator read it too.

The keys of F3

Key	Action
arrows	choose a colour
Enter	draw with it (back to F1)
e	edit it: up / down choose R, G or B; left / right change by 8; `<` and `>` by 1; Enter or Esc to finish
a	add a colour after the chosen one
Del	remove it (one colour stays at least)
[and]	move it
s	sort by hue
f	take the colours used in the sheet (the 256 used most)
1	the SDK's palette
2	bm Studio's palette
x	change the drawing colour into the chosen one, in the whole sprite
X	the same, in the whole sheet

Good to know

- The console keeps 16 bits per pixel (RGB565). Next to each colour, the screen shows how it looks on the console.
- A pixel you do not redraw keeps the colour it had in the file; a pixel you redraw takes the palette colour.

Caveats

- **X** does not work on a sheet bigger than 512x512: use **x** in the sprite.

Importing and exporting resources

What it does

A resource is one thing taken out of a cartridge in its own file. For pictures, the file is a `.bmi` : the sheet (or a piece of it), the zones of `SPRITES` , their `BOXES` and the tiles' `FLAGS` . A palette is a `.bmc` .

File	Holds
`.bmi`	images and sprites: sheet, SPRITES, BOXES, FLAGS
`.bmc`	a palette
`.bmk`	a kit: several kinds together on one shared sheet

What you can do today

- In the menu's **Lib** tab, the **Images** group lists the zones of SPRITES. `docs/RISORSE.md` describes **A** opening one in bm Pixel, but this was not confirmed in the code. The tab is hidden unless `lib_tab=1` is in `bm/config.txt` .
- On the PC, `scripts/bmres.py` extracts a sprite zone into a `.bmi` (with its boxes), adds a resource to a `.bm` , and converts to and from `.png` , `.hex` and `.gpl` .

COPY AND PASTE

```
python3 scripts/bmres.py list GAME.bm
python3 scripts/bmres.py extract GAME.bm --sprite coin -o COIN.bmi
python3 scripts/bmres.py add GAME.bm COIN.bmi
```

Caveats

- bm Pixel itself has no import or export command. [TO FILL: whether bm Pixel opens and saves `.bmi` and `.bmc` files directly; the notes plan it]
- Adding a resource to a `.bm` never removes anything, and a name that exists gets a number.

Guided tutorials

Draw your first sprite

1. Start bm Pixel from the **Dev** tab. If the list of files opens, press **Esc** to close it; the new empty sheet is already there. (Later, **Ctrl+N** makes another.)
2. Press **F1**. You have a new sheet of 256x256 and the SDK's palette.
3. Press **z** until the panel says `16x16`. Press **t** if you want the grid.
4. Press **.** a few times until you like a colour, or press **1** to **9, 0**. The panel shows `colour n`.
5. Press **b** for the pencil. Move the pointer with the arrows and press **space**. Hold space and move to draw a line of pixels.
6. Press **y** to draw mirrored: what you draw on one side appears on the other. Press **y** again to stop.
7. If you make a mistake, press **Ctrl+Z**.
8. Press **Esc**, choose **Save as...**, type a name such as `hero.bme` and press Enter. The screen says `saved`.

[TO FILL: shots/editor-pixel-first.png]

> **TRY IT**
> Draw a coin: a filled oval with **O**, then pick a darker colour and draw an outline with **o**.

Make an animated sprite

1. Open your sprite on F1 at size 16x16. It is the first frame.
2. Press **PgDn**: you are on the next sprite, to the right on the sheet. Draw the second frame.
3. Press **PgDn** again for a third frame, and so on. Frames must be next to each other, left to right.
4. Press **PgUp** until you are back on the first frame.
5. Press **+** until the panel says the number of frames you drew. The box on the right plays them.
6. Press **>** or **<** until the speed looks right. The line under the panel says the fps. **p** pauses.
7. Press **k** for onion skin: the frame before shows under the one you are drawing.
8. Save with **Ctrl+S**.

> **BEWARE**
> The number of frames and the fps you set are remembered by bm Pixel for this file, but they are not written into the cartridge. To use them in a game as a named sprite, see the chapter on zones.

Mark the tiles that are solid

1. In bm Pixel, draw your tile at size 8 (**z**): a wall, a floor. Save with **Ctrl+S**.
2. Open the project in bm SDK (the file keeps its sheet).
3. On the sprite page, choose the cell of the wall tile and turn flag 0 on (the keys **0** to **7** are for the flags).
4. In your code, read it: `fget(mget(cx, cy), 0)` is `true` for a solid cell.

The flags are in the table of the chapter on tile flags: 0 solid, 1 platform, 2 ladder, 3 water, 4 hurts.

> **NOTE**
> Step 3 is the one the notes describe in one line only. Check on your console how the SDK shows the flags, and see the gap in the chapter on tile flags.

Add a hitbox

1. Write a zone in your `zones.txt` and, under it, its boxes (see the chapter on hitboxes).
2. Build the cartridge with `mkbm.py --sprites zones.txt` and copy it to the card.
3. Open it in bm Pixel and go to F1. Press **z** until the size is the zone's frame size (16 for a 16x16 zone).
4. Go to the first frame of the zone with the arrows on F2, or with **PgUp** and **PgDn** on F1. The panel shows `zone "name" frame 1/n`.
5. Check that **j** says `hitboxes on (BOXES)`. The rectangles appear over the sprite: blue for hurt, red for hit, green for body.
6. Press **PgDn** to see the next frame's boxes. If a box is in the wrong place, change the numbers in `zones.txt` and build again.

> **TRY IT**
> Open the sheet page (F2), press **j** and look at all the boxes of all the zones at once.

Tips and limits

Limits

What	Limit
Sprite sizes	8, 16, 32, 64, 128 pixels
Sheet size	multiples of 8, from 8 up to 4096
New sheet	256x256
Palette	1 to 256 colours
Animation frames	1 to 16, speed 1 to 30 fps
Zones in a file	1 to 1024
Boxes in a file	1 to 4096
Files remembered	the last 6

Tips

- Keep the frames of one animation next to each other on the same row.
- Draw tiles at size 8 so one sprite is one tile.
- To change the sheet size: **Shift+R** on F2. What fits stays; the rest is cut.
- A big sheet takes a few seconds to save; the screen says `saving ...` .
- Use **x** and **X** on F3 to change one colour into another without redrawing.
- Try the game with **F5** often: you come back to the same page.

When something goes wrong

- `cannot open ...` : the file is not a valid `.bm` or `.bme` . Open another.
- `save failed: ...` : the message says why; try another name with **Save as...**.
- `unsaved changes: choose again ...` : choose the same entry again to confirm, or save first.

Open points

- **The pages**: picture to take: shots/editor-pixel-draw.png
- **The pages**: picture to take: shots/editor-pixel-sheet.png
- **Sprites, zones, frames and fps**: how to name a zone and give it frames and fps. The old notes say bm Pixel makes zones, but the code of F1, F2 and F3 has no key for it; today zones are written in a text file (`name x y width height [frames [fps
- **Tile flags**: the exact keys to set a tile's flags on the bm SDK sprite page (the notes say keys 0 to 7 turn the cell's flags on) and how the screen shows them
- **Hitboxes and hurtboxes**: picture to take: shots/editor-pixel-boxes.png
- **Hitboxes and hurtboxes**: how to create boxes on the console. Today they are written under the zone in the zones text file, with lines `hurt`, `hit` or `body`, the frame (`*` for every frame) and the rectangle x, y, width, height
- **Importing and exporting resources**: whether bm Pixel opens and saves `.bmi` and `.bmc` files directly; the notes plan it
- **Guided tutorials**: picture to take: shots/editor-pixel-first.png